UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL

FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

ACTIVIDAD 3. BÚSQUEDA LOCAL Y OPTIMIZACIÓN

Alumno: Eduardo Parrilla Toledano

Grupo:172

Docente: Mtro. Javier García Lara

1. Introducción

La búsqueda local intenta mejorar una solución modificándola paso a paso, sin necesidad de explorar todos los estados posibles. Es útil para problemas de optimización en los que se busca minimizar un costo o maximizar una función objetivo. En esta actividad se comparan Hill Climbing y Simulated Annealing.

2. Planteamiento del problema

Se desea encontrar el valor de x que minimiza la función f(x) = (x - 3)^2 + 2 en el intervalo [-10, 10]. El mínimo global está en x = 3 y el costo mínimo es 2. El programa usará una búsqueda discreta con pasos de 0.1.

- Estado: un valor candidato de x.
- Estado inicial: un valor elegido dentro del intervalo.
- Vecinos: valores x - paso y x + paso que permanezcan dentro del intervalo.
- Función objetivo: f(x) = (x - 3)^2 + 2.
- Criterio: seleccionar valores con menor costo; Simulated Annealing también puede aceptar temporalmente algunos peores.

3. Método A: Hill Climbing

Hill Climbing evalúa los vecinos de la solución actual y se mueve al vecino que mejora la función objetivo. Termina cuando ningún vecino mejora la solución. Es sencillo, pero puede quedarse atrapado en óptimos locales.

4. Método B: Simulated Annealing

Simulated Annealing utiliza una temperatura que disminuye gradualmente. Si una solución empeora el costo, puede aceptarla con probabilidad exp(-diferencia/temperatura). Al inicio explora más; al bajar la temperatura, se vuelve más selectivo. Esto puede ayudar a escapar de óptimos locales.

5. Código Python

```
import math
import random

random.seed(7)

def costo(x):
    return (x - 3) ** 2 + 2

def hill_climbing(inicio, paso=0.1, max_iter=5000):
    x = inicio
    for _ in range(max_iter):
        candidatos = [x]
        if x - paso >= -10:
            candidatos.append(x - paso)
        if x + paso <= 10:
            candidatos.append(x + paso)

        mejor = min(candidatos, key=costo)
        if costo(mejor) >= costo(x):
            break
        x = mejor
    return x, costo(x)
```

```
def simulated_annealing(inicio, paso=0.1,
                        temperatura=10.0,
                        enfriamiento=0.995,
                        max_iter=5000):
    x = inicio
    mejor_x = x
    t = temperatura

    for _ in range(max_iter):
        direccion = random.choice([-1, 1])
        candidato = x + direccion * paso

        if candidato < -10 or candidato > 10:
            t *= enfriamiento
            continue

        diferencia = costo(candidato) - costo(x)

        if diferencia < 0 or random.random() < math.exp(-diferencia / max(t, 1e-12)):
            x = candidato

        if costo(x) < costo(mejor_x):
            mejor_x = x

        t *= enfriamiento
        if t < 0.001:
            break

    return mejor_x, costo(mejor_x)

inicio = -8.0
x_hc, f_hc = hill_climbing(inicio)
x_sa, f_sa = simulated_annealing(inicio)

print("Hill Climbing: x =", round(x_hc, 2),
      "costo =", round(f_hc, 4))
print("Simulated Annealing: x =", round(x_sa, 2),
      "costo =", round(f_sa, 4))
print("Óptimo teórico: x = 3, costo = 2")
```

6. Resultados y análisis

El óptimo teórico de la función es x = 3, con costo 2. Hill Climbing debería acercarse a ese punto desde el inicio seleccionado, porque la función es convexa. Simulated Annealing también busca ese mínimo, pero su trayectoria puede variar debido a la aleatoriedad. Los valores exactos deben obtenerse ejecutando el código en Python.

Método	Resultado esperado	Ventaja	Limitación
Hill Climbing	Cerca de x=3, costo 2	Simple y rápido	Puede quedar en óptimos locales
Simulated Annealing	Cerca de x=3, costo 2	Puede escapar de óptimos locales	Depende de temperatura y aleatoriedad

7. Complejidad

Ambos algoritmos realizan como máximo un número limitado de iteraciones. Si cada iteración evalúa una cantidad constante de vecinos, la complejidad temporal es O(I), donde I es el

número de iteraciones, y la memoria auxiliar es O(1) para esta representación de una sola variable.

8. Conclusión

Hill Climbing mejora la solución en cada paso y es apropiado para problemas sencillos, mientras que Simulated Annealing permite aceptar temporalmente soluciones peores para explorar otras regiones. La elección depende de la función objetivo, el tiempo disponible y la calidad de solución requerida.

9. Referencias

- Russell, S., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.
- Python Software Foundation. (s. f.). random — Generate pseudo-random numbers. Python documentation. https://docs.python.org/3/library/random.html